

Primitive Datentypen: Grund-Datentypen von Java, aus denen komplexe Datentypen aufgebaut werden. String ist kein primitiver Datentyp, braucht man aber oft.

Ganze Zahlen (byte, short, int, long)

Dienen zur Darstellung der Zahlen von

$$\{-2^{8 \cdot n - 1}, \dots, 2^{8 \cdot n - 1} - 1\}, \text{ wobei:}$$

$$\underbrace{\{-2^7, \dots, 2^7 - 1\}}_{-128 \dots 127} \text{ byte: } n=1$$

$$\text{short: } n=2$$

$$\text{int: } n=4$$

$$\text{long: } n=8$$

Speicherplatz:
8 Bit = 1 Byte

16 Bit = 2 Byte

32 Bit = 4 Byte

64 Bit = 8 Byte

Zahlen werden im Zweierkomplement gespeichert.

Bei n Bits kann man die Zahlen von

$$-2^{n-1} \text{ bis } 2^{n-1} - 1 \text{ darstellen.}$$

Bei $n=3$: Zahlen von $\underbrace{-(2^2)}_{-4}$ bis $\underbrace{2^2 - 1}_3$

$$\begin{array}{|c|c|c|} \hline & 2^2 & 2^1 & 2^0 \\ \hline \end{array} \quad \text{Stelle die negative}$$

	2^2	2^1	2^0
3	0	1	1
2	0	1	0
1	0	0	1
0	0	0	0
-1	1	1	1
-2	1	1	0
-3	1	0	1
-4	1	0	0

Stelle die negative Zahl $-z$ als $2^m - z$ dar.
 z.B.: -4 wird dargestellt als $2^3 - 4 = 4$
 -3 wird dargestellt als $2^3 - 3 = 5$

- Vorderste Bit gibt Vorzeichen an ($1 \hat{=}$ negativ, $0 \hat{=}$ nicht-negativ)
- Von z zu $-z$: Invertiere Ziffern und addiere 1

z.B.: $z \hat{=}$

	0	1	0
Invertiere Ziffern	1	0	1
addiere 1	+ 0	0	1
	1	1	0

$\hat{=}$ -2

Arithmetische Operationen werden auf Binärzahlen im Zweierkomplement ausgeführt.

$\text{int } x = \underbrace{2\,147\,483\,647}_{2^{31}-1};$
 ← steht für 2.147.483.647

int y = 1;

x = x + y;

Was macht Java?

$$\begin{array}{r} 01\dots11 \\ + 0\dots01 \\ \hline 10\dots00 \end{array} \leftarrow \text{entspricht der Zahl } -2^{31}$$

Überlauf!

x hat den Wert -2_147_983_648

Quelle von Prog-Fehlern.

Schlechter Stil: Ausnutzen von Überläufen für
Prog.-Tricks

Operationen auf Ganzzahlen: +, -, *, /, %
Ganzzahldivision,
Rest wird weggelassen,
d.h. $2/3$ ergibt 0
Modulo,
d.h. Rest der
Division

Operationen auf byte, short, int erzeugen int:

byte a = 1;

a = a+1;
int

Fehler!

int

Werte vom Typ "long" lassen sich durch "L" erzeugen:

1L
1l

ist die Zahl 1 als "long" Wert

jshell mit Typen: /set feedback verbose

Gleitkommazahlen (float, double)

-1.5

100.
100,0

.57
0,57

1.2 e -23
 $1,2 \cdot 10^{-23}$

1.2f

1,2 als float-Zahl

1.2 oder 1.2d

1,2 als double-Zahl

	Vorzeichen	Exponent	Manisse	
float	1 Bit	8 Bit	23 Bit	} 32 Bit
double	1 Bit	11 Bit	52 Bit	} 64 Bit

Operationen: +, -, *, /, ...

Vorsicht: Rundungsfehler

Wahrheitswerte (boolean)

Datentyp mit Werten true, false.

Vordef. Operationen mit Ergebnis vom Typ boolean:

$=$, $!=$, $>$, $>=$, $<$, $<=$, ...
↑ Gleichheit ↑ Ungleichheit

Bsp: boolean $b = (2 == 3);$
 ↑ ↑
 Zuweisung Gleichheit
 false

Operationen auf Werten vom Typ boolean:

$\&\&$, $\|\|$, $!$
↑ ↑ ↓
und oder nicht
(Konjunktion) (Disjunktion) (Negation)

Bsp: false $\&\&$ true

$\&\&$ und $\|\|$ werden von links nach rechts abgearbeitet.
Wenn Ergebnis schon nach Auswertung des linken Arguments feststeht, dann wird das rechte Argument nicht ausgewertet.

$\left. \begin{array}{l} \text{false } \&\& \text{ exp} \\ \text{true } \|\| \text{ exp} \end{array} \right\} \begin{array}{l} \text{Hier wird der} \\ \text{Ausdruck exp} \\ \text{nicht ausgewertet.} \end{array}$ "expression"

true || exp) nicht ausgewertet. - expression

Zeichen (char) - 16 Bit

'a', 'A', '1', '\$', '\n'

↑
Steuerelemente für "newline"

Vordef. Operationen:

==, !=, <, <=, >, >=, ...

Vergleich der Zeichen anhand der Nummer in
Unicode:

Bsp: 'a' hat die Nr. 97

'b' hat die Nr. 98

'a' < 'b' ergibt true

⇒ Vergleich nach alphabetischer Sortierung

Zeichenkette (String)

kein primitiver Datentyp

- Notation mit "...": "hallo"
- Konkatination mit: "hal" + "lo" ergibt "hallo"

'a' Zeichen vom Typ char

"a" "abc" Strings

"", "a", "abc" Strings

Typkonversion

Java ist eine statisch getypte Sprache,
d.h. jeder Ausdruck hat einen Typ und
Operationen können nur auf Argumente des richtigen
Typs angewendet werden.

Typkorrektheit wird vor der Laufzeit vom Compiler
überprüft ("statisch").

(Dynamisches Typ-Checking z.B. in
JavaScript, Python, ...)

⇒ bei statisch getypten Sprachen:

keine Typüberprüfung zur Laufzeit nötig.

Manchmal möchte man Werte von einem Typ
in einen anderen Typ konvertieren:

2 + 3 ✓

+ verknüpft zwei int-Werte

2.3 + 3.5 ✓

+ verknüpft zwei double-Werte

2 + 3.5
int double

and erklärt:

Hier wird 2 in den

double-Wert 2.0 konvertiert.

⇒ 5.5 vom Typ double

Implizite Datentypanpassung

- vom speziellen zum allgemeinen Typ
- passiert automatisch, wenn Werte des allgemeineren Datentyps benötigt werden.
- Konversion zu String passiert nicht automatisch, sondern nur in bestimmten Funktionen wie IO.print oder +.

z.B. durch toString

int x = 'a';
int char

setzt x auf 97

int y = 'a' + 1;

setzt y auf 98

Explizite Datentypanpassung (Type Cast)

- Umwandlung von einem Typ zum anderen wird explizit erzeugen.
- Geht auch vom allgemeinen zum speziellen Typ.

int x = (int) 2.7;
double

setzt x auf 2

(int) 'a'

ergibt 97

(float) 1/2
1.0f
wird
automatisch in
2.0f konvertiert

ergibt 0.5f

(float) (1/2)

ergibt 0.0f

(char) ('a' + 1)
wird
konvertiert in 97

ergibt 'b'